SMART ANALYSIS REPORTER

Toolkit & System Design Report

System Documentation

Student Performance Prediction

Prepared by Kavin Ganapathy
Student ID: 100008820
Date: 2026-06-07

Table of Contents

1. Introduction & Purpose

1.1 About this document

This document describes the design and implementation of **Smart Analysis Reporter**, a web-based statistical analysis and reporting toolkit. Its focus is the **toolkit itself** — what each feature does, how a user drives it, what it computes, and the software design behind it. The statistical findings for the demonstration dataset are reported separately in the companion *Analysis Documentation*.

1.2 Problem the system solves

PROBLEM STATEMENT

Producing a statistical report normally means repeating the same manual steps — loading data, running tests, making charts, writing up findings — for every dataset. Smart Analysis Reporter packages those steps into reusable tools and a report engine, so analysis and reporting become repeatable and fast.

1.3 Intended audience

- Evaluators assessing the toolkit's capabilities and engineering.
- Future maintainers extending the analysis modules or adding features.
- Analysts who want to understand exactly what each tool computes.

1.4 Document map

Section 2 gives an abstract; Section 3 covers the system architecture and data flow; Section 4 the technology stack; Section 5 the codebase and module design; Section 6 is a granular catalogue of every feature; Section 7 details the report generation subsystem; Section 8 introduces the bundled demonstration dataset; and Section 9 concludes with future work.

2. Abstract

Smart Analysis Reporter is a layered, modular Python application with a Streamlit multipage front end and a dedicated report engine. It provides nine interactive analysis tools — data profiling, descriptive statistics, frequency tables, Q-Q normality plots, correlation, hypothesis testing, regression, time series — plus a report builder that turns any analysis into an editable, exportable document (PDF, Word, HTML). Analysis logic lives in pure-Python modules independent of the UI, which keeps the system testable, reusable and easy to extend.

3. System Architecture & Design

3.1 Architectural overview

The application is organised in four layers with a strict separation of concerns: a **data layer** that loads and caches the dataset; an **analysis layer** of pure-Python modules that implement the statistics; a **presentation layer** of Streamlit pages; and a **reporting layer** that renders charts and exports documents. Pages never contain statistical logic — they call modules — which means the same functions power both the interactive app and the generated documentation.

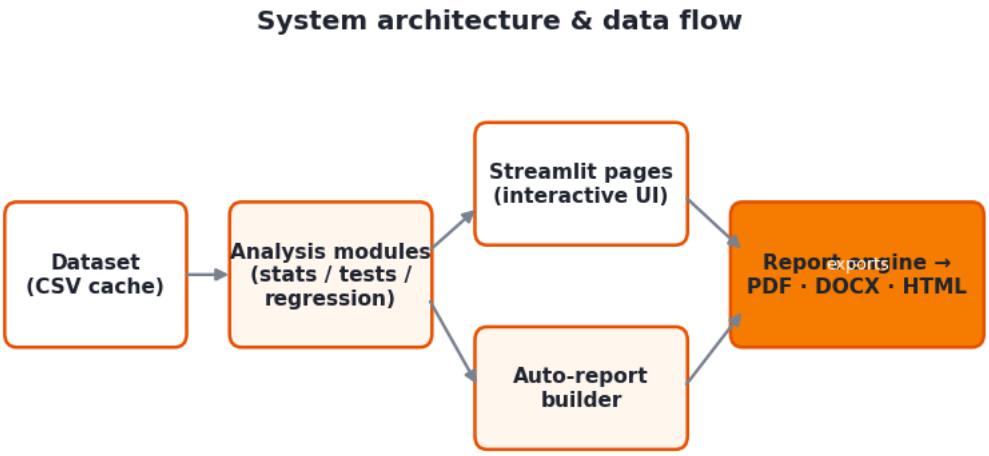

Figure 3.1 — System architecture and data flow.

3.2 Runtime data flow

- On first load, the dataset is read once and stored in Streamlit *session state* (key **df**), shared across every page.
- Selecting a page runs its script: it reads **df**, calls the relevant analysis module, and renders interactive Plotly charts and tables.
- An "■ Add to report" control renders a themed Matplotlib image of the current output, writes an auto-generated inference, and appends an item to the report state (key **report**).
- The Report Builder reads that state for editing; the export engine serialises it to PDF / DOCX / HTML.

3.3 State management

Two objects in session state hold all cross-page state: **df** (the active dataset) and **report** (a dictionary of a *cover* plus an ordered list of *items*). Because Streamlit re-runs a page top-to-bottom on every interaction, test results are also stashed in session state so they survive re-runs and remain available to add to the report.

3.4 Design principles

- **Modularity** — statistics live in importable modules, not in page scripts.
- **Reusability** — chart renderers and inference builders are shared by the app, the report engine and these documents.

- **Portability** — the demonstration dataset is cached in-repo, so the deployed app needs no external credentials.
- **Separation of UI and logic** — enabling headless testing of every module.

4. Technology Stack

The toolkit is built entirely in Python. Each dependency maps to a clear role:

Layer	Technology	Role in the toolkit
UI	Streamlit	Multipage interactive web UI, widgets and session state
Computation	pandas, NumPy	Data structures and vectorised computation
Statistics	SciPy, statsmodels	Statistical tests, distributions and OLS / ARIMA models
Interactive charts	Plotly	Interactive on-screen charts
Report charts	Matplotlib	Themed static charts embedded in exported documents
PDF export	ReportLab	PDF report and documentation generation
Word export	python-docx	Editable Word (.docx) generation
Data source	kagglehub	Fetching / caching the demonstration dataset

5. Codebase & Module Design

The application is split into a thin set of Streamlit page scripts (in **pages/**) and a library of analysis modules (in **modules/**). Each module owns one responsibility and exposes plain functions that take a pandas object and return numbers, data frames or chart images — never UI. This table maps every module to its responsibility:

Module	Responsibility
app.py	Home page — dataset loading, overview, one-click report trigger
ui.py	Theme (white/orange), page header, KPI cards, Plotly template
datasets.py	Cached loader for the default demonstration dataset
data_loader.py	File upload parsing and Metric/Ordinal/Nominal column classification
profiling.py	Dataset summary and per-column profiling
descriptive.py	Summary statistics (centre, spread, shape)
frequency.py	Categorical and grouped-metric frequency tables
normality.py	Q-Q plotting positions and Shapiro-Wilk normality
correlation.py	Pearson / Spearman / Kendall correlation matrices
tests.py	t-tests, ANOVA, chi-square, Z-tests and effect sizes
assumptions.py	Live test assumption checks (normality, variance, sample size)
regression.py	Simple and multiple OLS regression
timeseries.py	Time-series preparation and ARIMA forecast
report.py	Report state, themed chart rendering, PDF/DOCX/HTML export
autoreport.py	One-click full-analysis report builder
docgen.py	Analysis & System documentation generators

6. Feature Catalogue

This section documents each tool in the toolkit. For every feature it states the purpose, the controls the user drives, the methods and statistics computed, the outputs produced, and the module that implements it.

6.1 Home & Data Loading

- **Purpose.** Entry point of the app; makes a dataset available to every other tool and offers a one-click report.
- **User inputs / controls.** Upload a CSV/Excel file, or use the bundled default dataset; reset to default.
- **Methods & computation.** Files are parsed with pandas (read_csv / read_excel); columns are classified as Metric, Ordinal (low-cardinality integers) or Nominal by a dtype + cardinality heuristic. The dataset is held in session state and cached.
- **Outputs.** KPI cards (rows, columns, missing, duplicates), a data preview, a column-type table, and an “Auto-generate report” button.
- **Backing module.** modules/datasets.py, modules/data_loader.py (app.py)

6.2 Data Profiler

- **Purpose.** A one-glance data-quality and structure report.
- **User inputs / controls.** None — operates on the loaded dataset.
- **Methods & computation.** Computes row/column counts, total missing values and duplicate rows; per column it reports dtype, missing percentage, unique count and measurement level.
- **Outputs.** KPI cards plus a merged profile table (dtype + measurement level) and a preview.
- **Backing module.** modules/profiling.py, modules/data_loader.py

6.3 Descriptive Statistics

- **Purpose.** Summarise the centre, spread and shape of each metric variable.
- **User inputs / controls.** A variable selector to inspect one column in detail.
- **Methods & computation.** Mean, median, mode, variance, standard deviation, min/max/range, quartiles and IQR, plus skewness and kurtosis (pandas / SciPy). A fitted normal curve is overlaid on the histogram.
- **Outputs.** A summary table for all metric variables, KPI cards, a histogram with normal overlay, and a box plot.
- **Backing module.** modules/descriptive.py

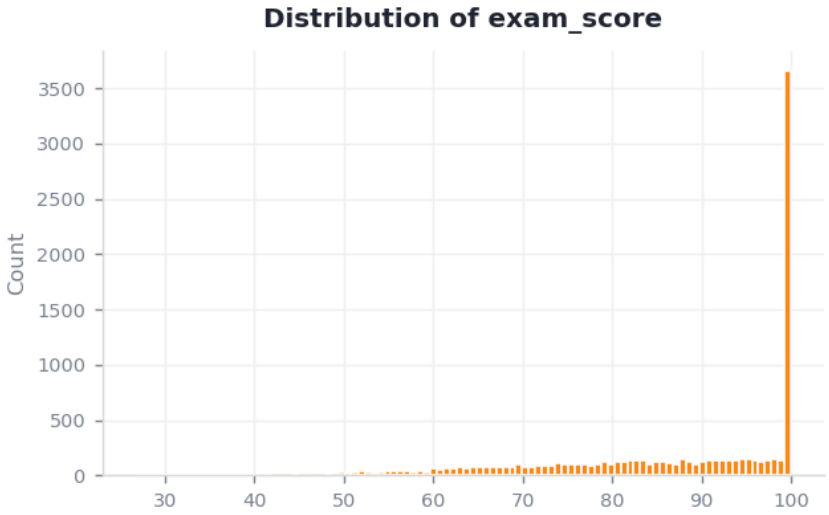

Figure 6.1 — Example output of the Descriptive tool: histogram with fitted normal curve.

6.4 Frequency Tables

- **Purpose.** Show how often each category or binned value occurs.
- **User inputs / controls.** A categorical variable, or a metric variable with a bin-count slider.
- **Methods & computation.** Absolute, relative (%) and cumulative-(%) frequencies via value counts; metric variables are binned with equal-width intervals (pd.cut).
- **Outputs.** A frequency table and an accompanying bar chart.
- **Backing module.** modules/frequency.py

6.5 Q-Q Plot & Normality

- **Purpose.** Test whether a variable is normally distributed — a key assumption for parametric tests.
- **User inputs / controls.** A metric variable.
- **Methods & computation.** Blom plotting positions are compared against theoretical normal quantiles (SciPy norm.ppf) with a Q1–Q3 reference line; a Shapiro-Wilk test and skewness / kurtosis quantify the departure from normality.
- **Outputs.** A Q-Q plot, a histogram with normal curve, the W statistic and p-value, KPI cards and a plain-language verdict.
- **Backing module.** modules/normality.py

Figure 6.2 — Example output of the Q-Q tool: sample quantiles against the normal reference line.

6.6 Correlation

- **Purpose.** Measure the linear association between every pair of metric variables.
- **User inputs / controls.** A correlation method — Pearson, Spearman or Kendall.
- **Methods & computation.** The full correlation matrix is computed and rendered as a colour heatmap; the strongest pair is summarised automatically.
- **Outputs.** A correlation-matrix table, a heatmap, and an auto-written interpretation.
- **Backing module.** modules/correlation.py

6.7 Hypothesis Testing

- **Purpose.** Decide whether observed differences or associations are statistically significant (α = 0.05).
- **User inputs / controls.** Choice of test (one-sample t, Welch independent t, paired t, ANOVA, chi-square, one- and two-sample Z); the relevant variables, hypothesised mean μ■ and σ.
- **Methods & computation.** Tests use SciPy; the independent t-test applies Welch's correction; effect sizes (Cohen's d, Cramér's V) and 95% confidence intervals are reported. A **live assumption engine** checks normality (Shapiro-Wilk), equal variances (Levene), sample-size adequacy (CLT) and expected-cell counts, shown as pass / review flags before the test is run.
- **Outputs.** Test statistic, p-value, effect size, contingency table (chi-square) and a plain-language verdict; results persist across re-runs and can be added to the report.
- **Backing module.** modules/tests.py, modules/assumptions.py

6.8 Regression

- **Purpose.** Quantify and model how predictors relate to an outcome.
- **User inputs / controls.** Simple linear: an X (predictor) and Y (outcome). Multiple OLS: a dependent variable and several independent variables.

- **Methods & computation.** Ordinary-least-squares fitting via statsmodels; the simple model reports intercept, slope, R^2 and the slope's p-value, and draws the fitted line; multiple OLS returns the full regression summary.
- **Outputs.** The fitted equation, KPI cards, an R^2 indicator, a scatter-plus-fit chart, and the OLS summary table.
- **Backing module.** modules/regression.py

Figure 6.3 — Example output of the Regression tool: data with the fitted least-squares line.

6.9 Time Series

- **Purpose.** Reveal a trend and produce a short forecast for date-bearing data.
- **User inputs / controls.** A date column and a value column.
- **Methods & computation.** Dates are parsed, sorted and indexed; an ARIMA(1,1,1) model produces a 30-step forecast (statsmodels). The tool gracefully reports when the dataset has no date column.
- **Outputs.** A trend line chart and a forecast table.
- **Backing module.** modules/timeseries.py

7. Report Generation Subsystem

The reporting layer is what makes the toolkit a *reporter*. It is built around a single state object and a set of renderers.

7.1 Report state model

The report is a dictionary held in session state with two parts: a **cover** (title, subtitle, author, ID, date and an editable executive summary) and an ordered list of **items**. Each item carries an id, a title, an editable inference, an optional chart image (PNG) and an optional table (data frame).

7.2 Add-to-report (incremental capture)

Every analysis page exposes an "■ Add to report" control. When used, the toolkit renders a themed Matplotlib image of the current chart, generates a one-sentence inference from the underlying statistics, and appends an item (de-duplicated by title). The user thus assembles a report while exploring.

7.3 Auto-report (one-click)

The auto-report builder chains the analysis modules to produce a complete report in one click — dataset overview, descriptive statistics, target distribution, spread and outliers, correlation, best-predictor regression, a categorical breakdown, a group comparison test and a normality check — each with an auto-written inference.

7.4 Report Builder page

- Edit the cover — title, subtitle, author, ID, date and executive summary.
- Edit each section's inference in place; reorder (↑/↓) or remove (✕) sections.
- Preview the orange cover banner live before exporting.

7.5 Export engine

A single report model is serialised three ways: a paginated **PDF** (ReportLab) with an orange cover and embedded charts/tables; an editable **Word** document (python-docx); and a self-contained **HTML** file with base64-embedded images. The same renderers power these companion documentation files.

7.6 Backing modules

modules/report.py (state, chart rendering, exporters), modules/autoreport.py (one-click builder), modules/docgen.py (these documents).

8. Demonstration Dataset

So the toolkit works out of the box, it bundles the **Student Performance Prediction** dataset (10,000 rows × 8 columns, target **exam_score**). It is used here only to illustrate the features — the full statistical study of this data is presented in the companion *Analysis Documentation*. The schema is:

Column	Type	Example	Unique
study_hours	Metric	7	11.000
attendance	Metric	56	61.000
sleep_hours	Metric	8	6.000
internet_usage	Metric	7	11.000
assignments_completed	Metric	10	21.000
previous_score	Metric	62	61.000
exam_score	Metric	100.0	3563.000
placement_status	Categorical	Placed	2.000

9. Project Conclusion

Smart Analysis Reporter packages a complete analytics workflow — from raw data, through a suite of interactive statistical tools, to a polished and editable report — behind a clean white-and-orange interface. Its layered design keeps statistics, UI and reporting independent, so new analyses, datasets or export formats can be added with minimal change, and every module can be tested headlessly.

9.1 Future work

- Multiple-regression and classification tools with model diagnostics.
- Automated outlier and data-quality reporting.
- Saved report templates and user accounts.
- Additional export themes and chart types.